

HPC exam project - part 1

A.Y. 2025-26

The damped wave equation describes the propagation of a wave in a medium where energy loss occurs over time. The attenuation of the wave is due, for example, to friction, viscosity, or resistance of the medium in which the wave propagates. Unlike the ideal wave equation, which assumes a perfectly conservative system, the damping term introduces a progressive decrease in the amplitude of the wave during propagation. The general partial differential equation that describes the phenomenon in space as time varies is the following:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u$$

Where $u(x, y, t)$ represents the height of the wave on a two-dimensional plane over time.

Implement a simulator which, given the values:

γ : wave damping

c : wave propagation speed

allows the wave propagation on the plane to be simulated. In practice, the simulator calculates the wave height for each point in the coordinate space (x, y) as time t changes. The simulation will generate an MP4 video; an example frame is shown below.



To perform the simulation, it is necessary to convert a continuous model (in which the function depends on real variables x, y and t) into a discrete model suitable for implementation on a computer. For performing this, a grid is introduced in both space and time:

- space is divided into equally spaced points along the x and y directions, with steps Δx and Δy ;
- time is discretized into instants separated by a time step Δt .

In this way, each point in space is identified by integer indices (i, j) , and time by an index n . The continuous function $u(x, y, t)$ is then replaced by a matrix of values $u_{i,j}^n$, which represents the value of the function at the grid point (i, j) at time step n .

Subsequently, the derivatives appearing in the differential equation are approximated using finite differences. In the literature, numerous techniques for discretizing a continuous function using **finite difference methods** are available.

After performing the appropriate transformations from the continuous to the discrete domain, a new equation is obtained that describes the evolution of the wave function in discrete space and discrete time. This discrete equation allows you to compute the $u_{i,j}^{n+1}$ value at a point in space with coordinates (i, j) at the next time step $(n+1)$, given the values at the same point at the current time step $(u_{i,j}^n)$ and previous time step $(u_{i,j}^{n-1})$.

Initially, the matrix $u_{i,j}^{n=0}$ (of size $M \times M$) is initialized to zero, i.e., all float values in the matrix are set to 0.0. In addition, the following wave parameters are defined:

γ : damping
 c : velocity

The following constants are also defined:

Δt : time integration step
 Δx : spatial integration step
 M : size of the square matrix
 N : number of simulation steps to compute

And the following boundary condition:

$u_{i_0,j_0}^{n=0}$: the initial impulse representing the start of the perturbation at time zero

The program, using OpenMP, performs the simulation by generating N matrices representing the evolution of $u_{i,j}^n$. Each matrix is saved in a separate file in Portable Graymap (PGM) format, within a dedicated folder named **sim**. Each time the computation of the matrix $u_{i,j}^{n+1}$ is completed, it is saved as a PGM file with the name **frame_<xxxxxx>.pgm**, where **xxxxxx** represents the current value of n written with 5 digits (for example, the value 1 is written as 00001).

PGM files are files that represent grayscale images; the first 3 lines of the file form the header and always contain the following values:

```
P5
M M
255
```

Where:

The first line specifies the pixel representation format; in particular, P5 indicates that each pixel is represented as a byte, with values ranging from 0 (black) to 255 (white). Intermediate values represent different shades of gray.

The second line indicates the number of pixels in the image; specifically, the first value indicates the number of rows, and the second indicates the number of columns. In our case, this corresponds to the dimensions of the matrix u .

The third line indicates the value for white, which is usually 255. Note that any pixel values greater than 255 will corrupt the image.

The following $M \times M$ lines after the PGM header contain the image pixels.

Once the N images are obtained, each representing a frame of the final video, it is possible to create the video by combining the individual frames using the external tool **FFmpeg**; the tool is free and can be downloaded into the program workspace using the following bash commands:

```
wget -O ffmpeg.tar.xz https://github.com/BtbN/FFmpeg-Builds/releases/download/latest/ffmpeg-master-latest-linux64-gpl.tar.xz
```

```
tar -xf ./ffmpeg.tar.xz -C ./ffmpeg_tool/
```

Note that it must be downloaded (using the **wget** command) and extracted (using the **tar** command) only once during the environment setup. Afterward, the video can be generated using the frames saved in the `sim` folder with the following command:

```
./ffmpeg_tool/ffmpeg-master-latest-linux64-gpl/bin/ffmpeg -framerate <XX> -i
./sim/frame_%05d.pgm -c:v libx264 -pix_fmt yuv420p nome_video.mp4
```

The **framerate** parameter indicates the number of frames per second and must be calculated based on the time integration step Δt used during the simulation.

Each student will find their own simulation parameters in file **Parameters_part_1.pdf** for their project.